How Amazon SageMaker Works

Amazon SageMaker is a fully managed service that enables you to quickly and easily integrate machine learning-based models into your applications. This section provides an overview of machine learning and explains how Amazon SageMaker works. If you are a first-time user of Amazon SageMaker, we recommend that you read the following sections in order:

Topics

- Machine Learning with Amazon SageMaker (p. 2)
- Explore and Preprocess Data (p. 4)
- Train a Model with Amazon SageMaker (p. 4)
- Deploy a Model in Amazon SageMaker (p. 7)

How It Works: Next Topic

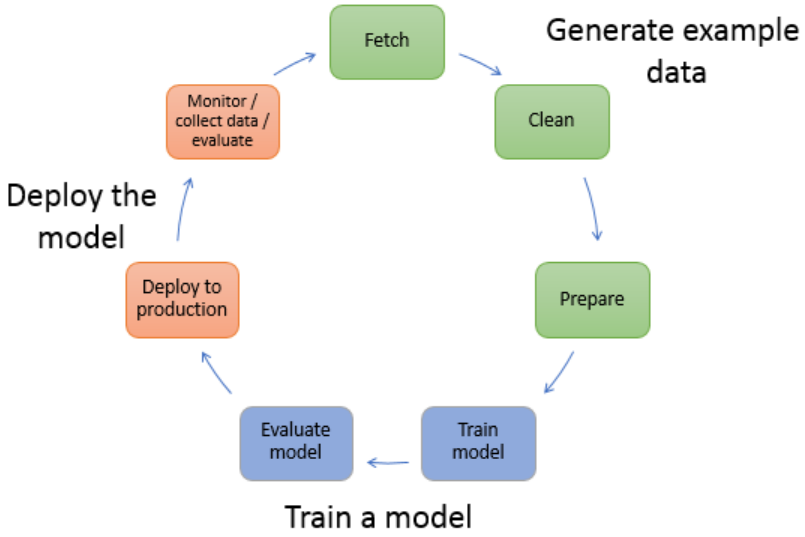
Machine Learning with Amazon SageMaker (p. 2)

Machine Learning with Amazon SageMaker

This section describes a typical machine learning workflow and summarizes how you accomplish those tasks with Amazon SageMaker.

In machine learning, you "teach" a computer to make predictions, or inferences. First, you use an algorithm and example data to train a model. Then you integrate your model into your application to generate inferences in real time and at scale. In a production environment, a model typically learns from millions of example data items and produces inferences in hundreds to less than 20 milliseconds.

The following diagram illustrates the typical workflow for creating a machine learning model:



As the diagram illustrates, you typically perform the following activities:

1. **Generate example data**—To train a model, you need example data. The type of data that you need depends on the business problem that you want the model to solve (the inferences that you want

the model to generate). For example, suppose that you want to create a model to predict a number given an input image of a handwritten digit. To train such a model, you need example images of handwritten numbers.

Data scientists often spend a lot of time exploring and preprocessing, or "wrangling," example data before using it for model training. To preprocess data, you typically do the following:

a. **Fetch the data**— You might have in-house example data repositories, or you might use datasets that are publicly available. Typically, you pull the dataset or datasets into a single repository.
b. **Clean the data**—To improve model training, inspect the data and clean it as needed. For example, if your data has a `country name` attribute with values `United States` and `US`, you might want to edit the data to be consistent.
c. **Prepare or transform the data**—To improve performance, you might perform additional data transformations. For example, you might choose to combine attributes. If your model predicts the conditions that require de-icing an aircraft, instead of using temperature and humidity attributes separately, you might combine those attributes into a new attribute to get a better model.

In Amazon SageMaker, you preprocess example data in a Jupyter notebook on your notebook instance. You use your notebook to fetch your dataset, explore it, and prepare it for model training. For more information, see Explore and Preprocess Data (p. 4). For more information about preparing data in AWS Marketplace, see data preparation.

2. **Train a model**—Model training includes both training and evaluating the model, as follows:
   - **Training the model**— To train a model, you need an algorithm. The algorithm you choose depends on a number of factors. For a quick, out-of-the-box solution, you might be able to use one of the algorithms that Amazon SageMaker provides. For a list of algorithms provided by Amazon SageMaker and related considerations, see Use Amazon SageMaker Built-in Algorithms (p. 56).

     You also need compute resources for training. Depending on the size of your training dataset and how quickly you need the results, you can use resources ranging from a single general-purpose instance to a distributed cluster of GPU instances. For more information, see Train a Model with Amazon SageMaker (p. 4).

   - **Evaluating the model**—After you've trained your model, you evaluate it to determine whether the accuracy of the inferences is acceptable. In Amazon SageMaker, you use either the AWS SDK for Python (Boto) or the high-level Python library that Amazon SageMaker provides to send requests to the model for inferences.

     You use a Jupyter notebook in your Amazon SageMaker notebook instance to train and evaluate your model.
3. **Deploy the model**— You traditionally re-engineer a model before you integrate it with your application and deploy it. With Amazon SageMaker hosting services, you can deploy your model independently, decoupling it from your application code. For more information, see Deploy a Model on Amazon SageMaker Hosting Services (p. 7).

Machine learning is a continuous cycle. After deploying a model, you monitor the inferences, collect "ground truth," and evaluate the model to identify drift. You then increase the accuracy of your inferences by updating your training data to include the newly collected ground truth. You do this by retraining the model with the new dataset. As more and more example data becomes available, you continue retraining your model to increase accuracy.

How It Works: Next Topic

Explore and Preprocess Data (p. 4)

Explore and Preprocess Data

Before using a dataset to train a model, data scientists typically explore and preprocess it. For example, in one of the exercises in this guide, you use the MNIST dataset, a commonly available dataset of handwritten numbers, for model training. Before you begin training, you transform the data into a format that is more efficient for training. For more information, see Step 4.3: Transform the Training Dataset and Upload It to Amazon S3 (p. 21).

To preprocess data use one of the following methods:

- Use a Jupyter notebook on an Amazon SageMaker notebook instance. You can also use the notebook instance to do the following:
  - Write code to create model training jobs
  - Deploy models to Amazon SageMaker hosting
  - Test or validate your models

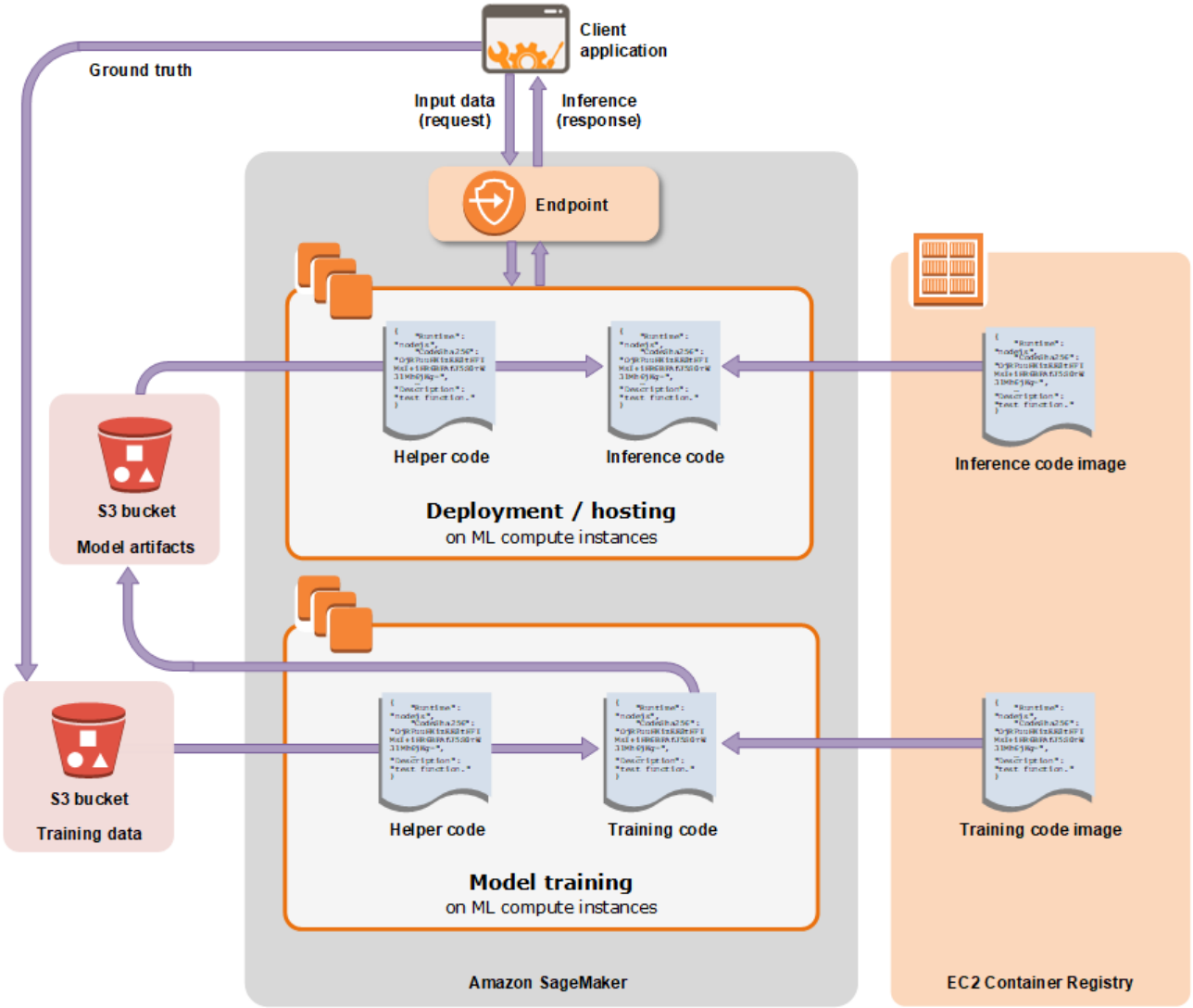
  For more information, see Use Notebook Instances (p. 36)
- You can use a model to transform data by using Amazon SageMaker batch transform. For more information, see Step 6.2: Deploy the Model with Batch Transform (p. 28).

How It Works: Next Topic

Train a Model with Amazon SageMaker (p. 4)

Train a Model with Amazon SageMaker

The following diagram shows how you train and deploy a model with Amazon SageMaker:



The area labeled Amazon SageMaker highlights the two components of Amazon SageMaker: model training and model deployment.

To train a model in Amazon SageMaker, you create a training job. The training job includes the following information:

- The URL of the Amazon Simple Storage Service (Amazon S3) bucket where you've stored the training data.
- The compute resources that you want Amazon SageMaker to use for model training. Compute resources are ML compute instances that are managed by Amazon SageMaker.
- The URL of the S3 bucket where you want to store the output of the job.
- The Amazon Elastic Container Registry path where the training code is stored. For more information, see Common Parameters for Built-In Algorithms (p. 58).

You have the following options for a training algorithm:

- **Use an algorithm provided by Amazon SageMaker**—Amazon SageMaker provides training algorithms. If one of these meets your needs, it's a great out-of-the-box solution for quick model

training. For a list of algorithms provided by Amazon SageMaker, see Use Amazon SageMaker Built-in Algorithms (p. 56). To try an exercise that uses an algorithm provided by Amazon SageMaker, see Get Started (p. 16).

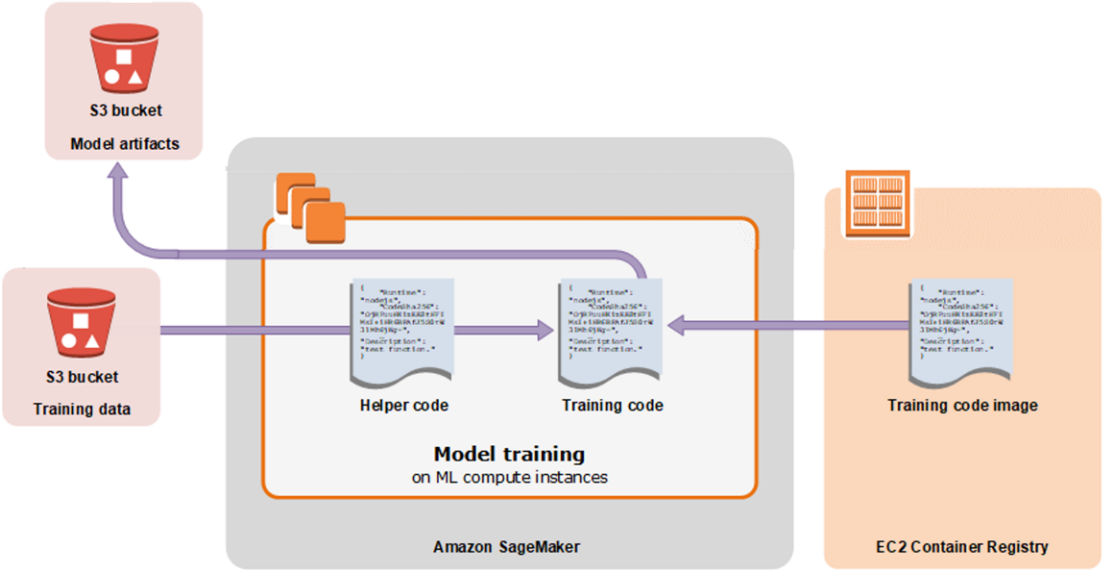
- **Use Apache Spark with Amazon SageMaker**—Amazon SageMaker provides a library that you can use in Apache Spark to train models with Amazon SageMaker. Using the library provided by Amazon SageMaker is similar to using Apache Spark MLLib. For more information, see Use Apache Spark with Amazon SageMaker (p. 440).
- **Submit custom code to train with deep learning frameworks**—You can submit custom Python code that uses TensorFlow or Apache MXNet for model training. For more information, see Use TensorFlow with Amazon SageMaker (p. 449) and Use Apache MXNet with Amazon SageMaker (p. 450).
- **Use your own custom algorithms**—Put your code together as a Docker image and specify the registry path of the image in an Amazon SageMaker `CreateTrainingJob` API call. For more information, see Use Your Own Algorithms or Models with Amazon SageMaker (p. 384).
- **Use an algorithm that you subscribe to from AWS Marketplace**—For information, see Find and Subscribe to Algorithms and Model Packages on AWS Marketplace (p. 431).

After you create the training job, Amazon SageMaker launches the ML compute instances and uses the training code and the training dataset to train the model. It saves the resulting model artifacts and other output in the S3 bucket you specified for that purpose.

You can create a training job with the Amazon SageMaker console or the API. For information about creating a training job with the API, see the CreateTrainingJob (p. 667) API.

When you create a training job with the API, Amazon SageMaker replicates the entire dataset on ML compute instances by default. To make Amazon SageMaker replicate a subset of the data on each ML compute instance, you must set the `S3DataDistributionType` field to `ShardedByS3Key`. You can set this field using the low-level SDK. For more information, see `S3DataDistributionType` in S3DataSource (p. 994).

> **Important**
> To prevent your algorithm container from contending for memory, you should reserve some memory for Amazon SageMaker critical system processes on your ML compute instances. If the algorithm container is allowed to use memory needed for system processes, it can trigger a system failure.



How It Works: Next Topic

Deploy a Model in Amazon SageMaker (p. 7)

Deploy a Model in Amazon SageMaker

After you train your model, you can deploy it to get predictions in one of two ways:

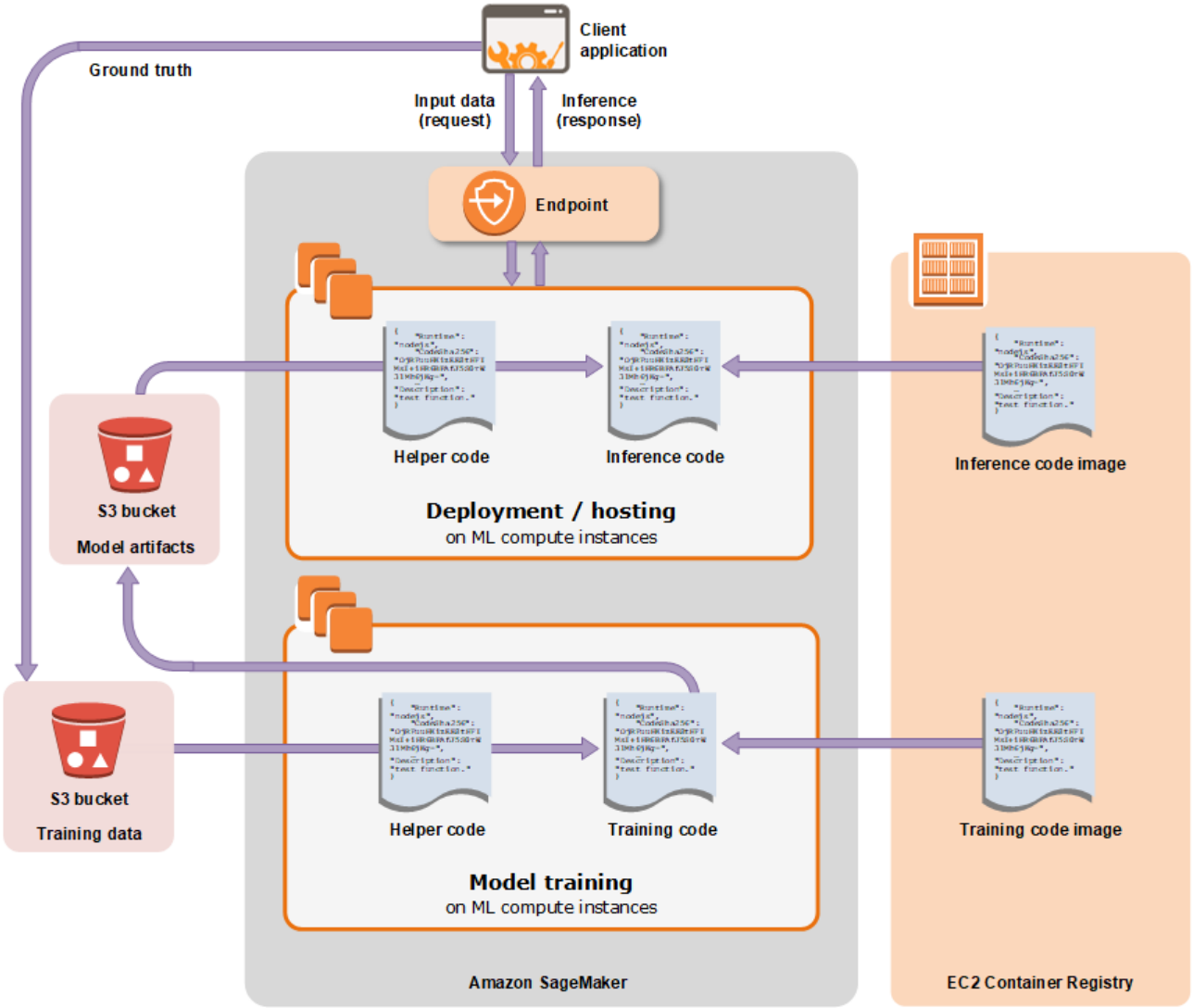
- To set up a persistent endpoint to get one prediction at a time, use Amazon SageMaker hosting services.
- To get predictions for an entire dataset, use Amazon SageMaker batch transform.

Topics

- Deploy a Model on Amazon SageMaker Hosting Services (p. 7)
- Get Inferences for an Entire Dataset with Batch Transform (p. 10)
- Validate a Machine Learning Model (p. 11)
- The Amazon SageMaker Programming Model (p. 12)

Deploy a Model on Amazon SageMaker Hosting Services

Amazon SageMaker also provides model hosting services for model deployment, as shown in the following diagram. Amazon SageMaker provides an HTTPS endpoint where your machine learning model is available to provide inferences.



Deploying a model using Amazon SageMaker hosting services is a three-step process:

1. **Create a model in Amazon SageMaker**—By creating a model, you tell Amazon SageMaker where it can find the model components. This includes the S3 path where the model artifacts are stored and the Docker registry path for the image that contains the inference code. In subsequent deployment steps, you specify the model by name. For more information, see the CreateModel (p. 648) API.
2. **Create an endpoint configuration for an HTTPS endpoint**—You specify the name of one or more models in production variants and the ML compute instances that you want Amazon SageMaker to launch to host each production variant.

   When hosting models in production, you can configure the endpoint to elastically scale the deployed ML compute instances. For each production variant, you specify the number of ML compute instances that you want to deploy. When you specify two or more instances, Amazon SageMaker launches them in multiple Availability Zones. This ensures continuous availability. Amazon SageMaker manages deploying the instances. For more information, see the CreateEndpointConfig (p. 635) API.
3. **Create an HTTPS endpoint**—Provide the endpoint configuration to Amazon SageMaker. The service launches the ML compute instances and deploys the model or models as specified in the configuration. For more information, see the CreateEndpoint (p. 632) API. To get inferences from

the model, client applications send requests to the Amazon SageMaker Runtime HTTPS endpoint. For more information about the API, see the InvokeEndpoint (p. 853) API.

Note
When you create an endpoint, Amazon SageMaker attaches an Amazon EBS storage volume to each ML compute instance that hosts the endpoint. The size of the storage volume depends on the instance type. For a list of instance types that Amazon SageMaker hosting service supports, see AWS Service Limits. For a list of the sizes of the storage volumes that Amazon SageMaker attaches to each instance, see Hosting Instance Storage Volumes (p. 381).

To increase a model's accuracy, you might choose to save the user's input data and ground truth, if available, as part of the training data. You can then retrain the model periodically with a larger, improved training dataset.

Best Practices for Deploying Models on Amazon SageMaker Hosting Services

When hosting models using Amazon SageMaker hosting services, consider the following:

- Typically, a client application sends requests to the Amazon SageMaker HTTPS endpoint to obtain inferences from a deployed model. You can also send requests to this endpoint from your Jupyter notebook during testing.

- You can deploy a model trained with Amazon SageMaker to your own deployment target. To do that, you need to know the algorithm-specific format of the model artifacts that were generated by model training. For more information about output formats, see the section corresponding to the algorithm you are using in Training Data Formats (p. 65).

- You can deploy multiple variants of a model to the same Amazon SageMaker HTTPS endpoint. This is useful for testing variations of a model in production. For example, suppose that you've deployed a model into production. You want to test a variation of the model by directing a small amount of traffic, say 5%, to the new model. To do this, create an endpoint configuration that describes both variants of the model. You specify the `ProductionVariant` in your request to the `CreateEndPointConfig`. For more information, see ProductionVariant (p. 981).

- You can configure a `ProductionVariant` to use Application Auto Scaling. For information about configuring automatic scaling, see Automatically Scale Amazon SageMaker Models (p. 365).

- You can modify an endpoint without taking models that are already deployed into production out of service. For example, you can add new model variants, update the ML Compute instance configurations of existing model variants, or change the distribution of traffic among model variants. To modify an endpoint, you provide a new endpoint configuration. Amazon SageMaker implements the changes without any downtime. For more information see, UpdateEndpoint (p. 840) and UpdateEndpointWeightsAndCapacities (p. 842).

- Changing or deleting model artifacts or changing inference code after deploying a model produces unpredictable results. If you need to change or delete model artifacts or change inference code, modify the endpoint by providing a new endpoint configuration. Once you provide the new endpoint configuration, you can change or delete the model artifacts corresponding to the old endpoint configuration.

- If you want to get inferences on entire datasets, consider using batch transform as an alternative to hosting services. For information, see Get Inferences for an Entire Dataset with Batch Transform (p. 10)

How It Works: Next Topic

Validate a Machine Learning Model (p. 11)

Get Inferences for an Entire Dataset with Batch Transform

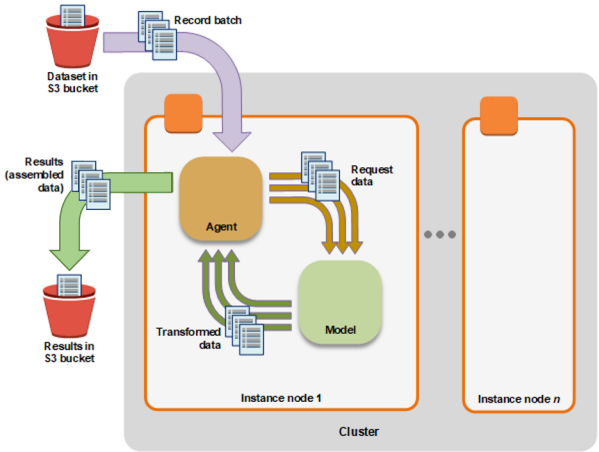
To get inferences for an entire dataset, use batch transform. With batch transform, you create a batch transform job using a trained model and the dataset, which must be stored in Amazon S3. Amazon SageMaker saves the inferences in an S3 bucket that you specify when you create the batch transform job. Batch transform manages all of the compute resources required to get inferences. This includes launching instances and deleting them after the batch transform job has completed. Batch transform manages interactions between the data and the model with an object within the instance node called an agent.

Use batch transform when you:

- Want to get inferences for an entire dataset and index them to serve inferences in real time
- Don't need a persistent endpoint that applications (for example, web or mobile apps) can call to get inferences
- Don't need the subsecond latency that Amazon SageMaker hosted endpoints provide

You can also use batch transform to preprocess your data before using it to train a new model or generate inferences.

The following diagram shows the workflow of a batch transform job:



To perform a batch transform, create a batch transform job using either the Amazon SageMaker console or the API. Provide the following:

- The path to the S3 bucket where you've stored the data that you want to transform.
- The compute resources that you want Amazon SageMaker to use for the transform job. *Compute resources* are machine learning (ML) compute instances that are managed by Amazon SageMaker.

- The path to the S3 bucket where you want to store the output of the job.
- The name of the Amazon SageMaker model that you want to use to create inferences. You must use a model that you have already created either with the CreateModel (p. 648) operation or the console.

The following is an example of what a dataset file might look like.

```
An example of input file content:
                Record1-Attribute1, Record1-Attribute2, Record1-Attribute3, ..., Record1-
AttributeM
                Record2-Attribute1, Record2-Attribute2, Record2-Attribute3, ..., Record2-
AttributeM
                Record3-Attribute1, Record3-Attribute2, Record3-Attribute3, ..., Record3-
AttributeM
                ...
                RecordN-Attribute1, RecordN-Attribute2, RecordN-Attribute3, ..., RecordN-
AttributeM
```

A record is a single input data unit, for information on how to delimit records for batch transform jobs, see `SplitType` in TransformInput (p. 1024).

For an example of how to use batch transform, see Step 6.2: Deploy the Model with Batch Transform (p. 28).

How It Works: Next Topic

Validate a Machine Learning Model (p. 11)

Validate a Machine Learning Model

After training a model, evaluate it to determine whether its performance and accuracy allow you to achieve your business goals. You might generate multiple models using different methods and evaluate each. For example, you could apply different business rules for each model, and then apply various measures to determine each model's suitability. You might consider whether your model needs to be more sensitive than specific (or vice versa).

You can evaluate your model using historical data (offline) or live data:

- **Offline testing**—Use historical, not live, data to send requests to the model for inferences.

  Deploy your trained model to an alpha endpoint, and use historical data to send inference requests to it. To send the requests, use a Jupyter notebook in your Amazon SageMaker notebook instance and either the AWS SDK for Python (Boto) or the high-level Python library provided by Amazon SageMaker.

- **Online testing with live data**—Amazon SageMaker supports deploying multiple models (called production variants) to a single Amazon SageMaker endpoint. You configure the production variants so that a small portion of the live traffic goes to the model that you want to validate. For example, you might choose to send 10% of the traffic to a model variant for evaluation. After you are satisfied with the model's performance, you can route 100% traffic to the updated model.

For more information, see articles and books about how to evaluate models, for example, Evaluating Machine Learning Models.

Options for offline model evaluation include:

- **Validating using a "holdout set"**—Machine learning practitioners often set aside a part of the data as a "holdout set." They don't use this data for model training.

  With this approach, you evaluate how well your model provides inferences on the holdout set. You then assess how effectively the model generalizes what it learned in the initial training, as opposed to using model "memory." This approach to validation gives you an idea of how often the model is able to infer the correct answer.

  In some ways, this approach is similar to teaching elementary school students. First, you provide them with a set of examples to learn, and then test their ability to generalize from their learning. With homework and tests, you pose problems that were not included in the initial learning and determine whether they are able to generalize effectively. Students with perfect memories could memorize the problems, instead of learning the rules.

  Typically, the holdout dataset is of 20-30% of the training data.

- **k-fold validation**—In this validation approach, you split the example dataset into *k* parts. You treat each of these parts as a holdout set for *k* training runs, and use the other *k*-1 parts as the training set for that run. You produce *k* models using a similar process, and aggregate the models to generate your final model. The value *k* is typically in the range of 5-10.

How It Works: Next Topic

The Amazon SageMaker Programming Model (p. 12)

The Amazon SageMaker Programming Model

Amazon SageMaker provides APIs that you can use to create and manage notebook instances and train and deploy models. For more information, see API Reference (p. 616).

Making API calls directly from code is cumbersome, and requires you to write code to authenticate your requests. Amazon SageMaker provides the following alternatives:

- **Use the Amazon SageMaker console**—With the console, you don't write any code. You use the console UI to start model training or deploy a model. The console works well for simple jobs, where you use a built-in training algorithm and you don't need to preprocess training data.

- **Modify the example Jupyter notebooks**—Amazon SageMaker provides several Jupyter notebooks that train and deploy models using specific algorithms and datasets. Start with a notebook that has a suitable algorithm and modify it to accommodate your data source and specific needs.

- **Write model training and inference code from scratch**—Amazon SageMaker provides both an AWS SDK and a high-level Python library that you can use in your code to start model training jobs and deploy the resulting models.

  - **The high-level Python library**—The Python library simplifies model training and deployment. In addition to authenticating your requests, the library abstracts platform specifics by providing simple methods and default parameters. For example:

- To deploy your model, you call only the `deploy()` method. The method creates an Amazon SageMaker model, an endpoint configuration, and an endpoint.

- If you use a custom framework script for model training, you call the `fit()` method. The method creates a .gzip file of your script, uploads it to an Amazon S3 location, and then runs it for model training, and other tasks. For more information, see Use Machine Learning Frameworks with Amazon SageMaker (p. 440).

- **The AWS SDK** —The SDKs provide methods that correspond to the Amazon SageMaker API (see Actions (p. 616)). Use the SDKs to programmatically start a model training job and host the model in Amazon SageMaker. SDK clients authenticate your requests by using your access keys, so you don't need to write authentication code. They are available in multiple languages and platforms. For more information, see SDKs.

In Get Started (p. 16), you train and deploy a model using an algorithm provided by Amazon SageMaker. That exercise shows how to use both of these libraries. For more information, see Get Started (p. 16).

- **Integrate Amazon SageMaker into your Apache Spark workflow**—Amazon SageMaker provides a library for calling its APIs from Apache Spark. With it, you can use Amazon SageMaker-based estimators in an Apache Spark pipeline. For more information, see Use Apache Spark with Amazon SageMaker (p. 440).

How It Works: Next Topic

Get Started (p. 16)